Kiến trúc Hexagonal

Kiến trúc mã nguồn Backend (Java + Spring Boot)

- 1. Lựa chọn Kiến trúc: Hexagonal Architecture
  - Tại sao chọn Hexagonal?
  - So sánh với các kiến trúc khác
- 2. Các lớp trong kiến trúc
  - A. Domain Layer
  - B. Application Layer
  - C. Infrastructure Layer
- 3. Cấu trúc Package mẫu
- 4. Minh họa Code
  - Ví dụ 1: Domain Entity (Profile.java)
  - Ví dụ 2: Command Object (CreateUserCommand.java)
  - Ví dụ 3: Use Case implementation (AuthenticationService.java)
  - Ví dụ 4: MapStruct Mapper (UserMapper.java)
- 5. Quy ước chung (Naming & Coding)
  - Naming Conventions
  - Lombok
  - Dependency Injection
  - MapStruct
- 6. Request Context Pattern
- 7. Ưu & Nhược điểm
- 8. Tài liệu Tham khảo

Kiến trúc mã nguồn Backend (Java + Spring Boot)

Tài liệu này định nghĩa cách cấu trúc source code cho các microservice trong hệ thống Financial Pulse. Chúng ta hướng tới sự tách biệt rõ ràng giữa logic nghiệp vụ (Domain) và chi tiết kỹ thuật (Infrastructure).

1. Lựa chọn Kiến trúc: Hexagonal Architecture

Chúng ta áp dụng **Hexagonal Architecture** (còn gọi là Ports and Adapters) kết hợp với tư duy **Domain-Driven Design (DDD-lite)**.

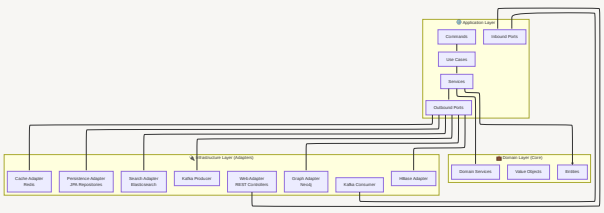
Tại sao chọn Hexagonal?

- **Độc lập framework**: Logic nghiệp vụ không bị phụ thuộc vào Spring Boot hay bất kỳ thư viện bên thứ ba nào.
- **Dễ dàng Unit Test**: Có thể test domain logic mà không cần database hay môi trường mạng.
- **Linh hoạt**: Dễ dàng thay đổi adapter (ví dụ: chuyển từ MySQL sang HBase) mà không phải sửa code nghiệp vụ.

So sánh với các kiến trúc khác

- **Layered (3-tier)**: Đơn giản nhưng dễ bị "leak" logic database vào Service layer (Anemic Domain Model).
- **Clean Architecture**: Khá tương đồng với Hexagonal nhưng có thể gây over-engineering nếu áp dụng quá máy móc các lớp (Entities, Use Cases, Presenters).

2. Các lớp trong kiến trúc



- **Driving Side** = *bên CHỦ ĐỘNG gọi hệ thống*
- **Driven Side** = *bên BỊ ĐỘNG được hệ thống gọi ra ngoài*

A. Domain Layer

Chứa "linh hồn" của ứng dụng. Tuyệt đối không import bất kỳ class nào từ `infrastructure` hay `interfaces`.

- **Entities**: Các đối tượng có định danh (vd: `Profile`, `Segment`).
- **Value Objects**: Đối tượng không có định danh, định nghĩa bằng thuộc tính (vd: `Address`, `Money`).
- **Domain Services**: Chứa logic nghiệp vụ liên quan đến nhiều Entities.

B. Application Layer

Đóng vai trò điều phối (Orchestration).

- **Commands**: Đối tượng chứa dữ liệu đầu vào cho các use case (vd: `CreateProfileCommand`, `LoginCommand`).
- **Use Cases (Input Ports)**: Interface định nghĩa những gì hệ thống có thể làm (vd: `CreateProfileUseCase`, `AuthenticationUseCase`).
- **Output Ports**: Interface mà domain cần để tương tác với thế giới bên ngoài (vd: `UserRepositoryPort`, `EventPublisherPort`, `CachePort`).
- **Services**: Triển khai các Use Cases, điều phối Domain Models và gọi Output Ports.
- **Context**: `RequestContext` và `RequestContextHolder` để lưu trữ thông tin request-scoped (requestId, userId, workspaceId).

C. Infrastructure Layer

Chứa các triển khai cụ thể (Implementation) của các Ports.

- **Persistence Adapter**: JPA Repositories cho MySQL, HBase client cho event storage.
- **Messaging Adapter**: Kafka producer và consumer sử dụng native `kafka-clients`.

- **Cache Adapter**: Redis client sử dụng Lettuce (async, non-blocking).
- **Search Adapter**: Elasticsearch client sử dụng official Elasticsearch Java client.
- **Graph Adapter**: Neo4j driver cho graph queries.
- **API/Web Adapter**: Spring MVC Controllers, DTOs, Filters, Exception Handlers.
- **Mappers**: MapStruct interfaces để chuyển đổi giữa Domain Models và JPA Entities.
- **Config**: Spring Configuration classes cho các clients và beans.
- **Bootstrap**: Components khởi tạo dữ liệu ban đầu (vd: `SystemAdminDataInitializer`).

3. Cấu trúc Package mẫu

Hệ thống sử dụng cấu trúc package theo Hexagonal Architecture với prefix `com.vcc.bigdata.cdp.layer`:

```
com.vcc.bigdata.financial.layer
├── domain
│   ├── model           (Entities, Value Objects)
│   └── service         (Domain Services)
├── application
│   ├── command         (Command objects cho use cases)
│   ├── context         (RequestContext, RequestContextHolder)
│   ├── port
│   │   ├── in          (Use Case interfaces)
│   │   └── out         (Repository/Client interfaces)
│   ├── query           (Query objects - nếu có)
│   └── service         (Use Case implementations)
├── infrastructure
│   ├── adapter
│   │   ├── in
│   │   │   ├── kafka   (Kafka Consumer Adapters)
│   │   │   └── web
│   │   │       ├── controller   (REST Controllers)
│   │   │       ├── dto          (Request/Response DTOs)
│   │   │       ├── exception    (Exception Handlers)
│   │   │       ├── filter       (HTTP Filters)
│   │   │       ├── security     (Security Filters)
│   │   │       └── util         (Web Utilities)
│   │   └── out
│   │       ├── cache           (Redis Cache Adapter)
│   │       ├── elasticsearch   (Elasticsearch Adapter)
│   │       ├── hbase           (HBase Adapter)
```

```
│   │       ├── jwt             (JWT Generator/Verifier)
│   │       ├── kafka           (Kafka Producer Adapter)
│   │       ├── neo4j           (Neo4j Graph Adapter)
│   │       └── persistence
│   │           ├── jpa
│   │           │   ├── entity      (JPA Entities)
│   │           │   └── repository  (JPA Repositories)
│   │           ├── mapper          (MapStruct Mappers)
│   │           └── *Repository.java (Repository Adapters)
│   ├── bootstrap       (Data Initializers)
│   └── config          (Spring Configuration)
└── Application.java    (Main Spring Boot Application)
```

4. Minh họa Code

Ví dụ 1: Domain Entity (Profile.java)

Dùng tư duy DDD, Entity tự quản lý trạng thái của mình.

```java
package com.company.cdp.profile.domain.model;

import lombok.Getter;
import java.util.Map;

@Getter
public class Profile {
    private final String profileId;
    private String email;
    private Map<String, Object> attributes;

    public Profile(String profileId, String email) {
        if (email == null || !email.contains("@")) {
            throw new IllegalArgumentException("Email không h
ợp lệ");
        }
        this.profileId = profileId;
        this.email = email;
    }
```

```
    public void updateAttribute(String key, Object value) {
        this.attributes.put(key, value);
    }
}
```

Ví dụ 2: Command Object (CreateUserCommand.java)

Encapsulate dữ liệu đầu vào cho use case.

```
package com.vcc.bigdata.cdp.layer.application.command;

import lombok.Getter;
import lombok.RequiredArgsConstructor;

@Getter
@RequiredArgsConstructor
public class CreateUserCommand {
    private final String email;
    private final String password;
}
```

Ví dụ 3: Use Case implementation (AuthenticationService.java)

Sử dụng **Constructor Injection** và Command objects.

```
package com.vcc.bigdata.cdp.layer.application.service;

import com.vcc.bigdata.cdp.layer.application.command.Register
UserCommand;
import com.vcc.bigdata.cdp.layer.application.port.in.Authenti
cationUseCase;
import com.vcc.bigdata.cdp.layer.application.port.out.UserRep
ositoryPort;
```

```java
import lombok.RequiredArgsConstructor;
import org.springframework.stereotype.Service;

@Service
@RequiredArgsConstructor
public class AuthenticationService implements AuthenticationU
seCase {
    private final UserRepositoryPort userRepositoryPort; // O
utput Port

    @Override
    public void register(RegisterUserCommand command) {
        // Business logic here
        User user = User.builder()
            .email(command.getEmail())
            .password(command.getPassword())
            .build();
        userRepositoryPort.save(user);
    }
}
```

Ví dụ 4: MapStruct Mapper (UserMapper.java)

Sử dụng MapStruct để giảm boilerplate code cho mapping giữa Domain Model và JPA Entity.

```java
package com.vcc.bigdata.cdp.layer.infrastructure.adapter.out.
persistence.mapper;

import org.mapstruct.Mapper;
import org.mapstruct.Mapping;
import org.mapstruct.factory.Mappers;
import com.vcc.bigdata.cdp.layer.domain.model.User;
import com.vcc.bigdata.cdp.layer.infrastructure.adapter.out.p
ersistence.jpa.entity.UserEntity;
```

```java
import com.vcc.bigdata.cdp.util.TimeUtils;

@Mapper(componentModel = "spring", uses = { TimeUtils.class
})
public interface UserMapper {
    UserMapper INSTANCE = Mappers.getMapper(UserMapper.clas
s);

    @Mapping(target = "id", expression = "java(UUID.fromStrin
g(entity.getId()))")
    @Mapping(target = "createdAt", source = "createdAt", qual
ifiedByName = "toInstant")
    @Mapping(target = "updatedAt", source = "updatedAt", qual
ifiedByName = "toInstant")
    User toDomainModel(UserEntity entity);

    @Mapping(target = "id", expression = "java(user.getId().t
oString())")
    @Mapping(target = "createdAt", source = "createdAt", qual
ifiedByName = "toTimestamp")
    @Mapping(target = "updatedAt", source = "updatedAt", qual
ifiedByName = "toTimestamp")
    UserEntity toEntity(User user);
}
```

5. Quy ước chung (Naming & Coding)

Naming Conventions

- **Class**: PascalCase (vd: `UserService`, `UserRepositoryPort`).
- **Method/Variable**: camelCase (vd: `findById`, `userRepository`).
- **Port In**: `<Action>UseCase` (vd: `AuthenticationUseCase`, `CreateUserUseCase`).

- **Port Out**: `<Resource>RepositoryPort`, `<Resource>Port`, `<Action>Port` (vd: `UserRepositoryPort`, `CachePort`, `EventPublisherPort`).
- **Command**: `<Action><Resource>Command` (vd: `CreateUserCommand`, `LoginCommand`).
- **Adapter**: `<Technology><Purpose>Adapter` (vd: `RedisCacheAdapter`, `KafkaEventPublisherAdapter`).
- **Mapper**: `<DomainModel>Mapper` (vd: `UserMapper`, `WorkspaceMapper`).

Lombok

- Chỉ dùng `@Getter`, `@Setter`, `@RequiredArgsConstructor`, `@Builder`.
- Hạn chế `@Data` hoặc `@Value` cho các Entity phức tạp để tránh side-effect.
- Sử dụng `@Builder` cho Domain Models và DTOs khi cần.

Dependency Injection

- Luôn dùng **Constructor Injection** với `@RequiredArgsConstructor`.
- Không dùng `@Autowired` field injection.

MapStruct

- Sử dụng MapStruct cho tất cả mapping giữa Domain Models và JPA Entities.
- Đặt mappers trong `infrastructure/adapter/out/persistence/mapper/`.
- Sử dụng `componentModel = "spring"` để tích hợp với Spring.
- Sử dụng `@Named` methods trong utility classes (vd: `TimeUtils`) cho custom mappings.

6. Request Context Pattern

Hệ thống sử dụng `RequestContext` và `RequestContextHolder` để lưu trữ thông tin request-scoped:

- **RequestContext**: Chứa `requestId`, `userId`, `workspaceId`.
- **RequestContextHolder**: Sử dụng `ThreadLocal` để lưu trữ context cho mỗi request.

- **Usage**: Truy cập context từ bất kỳ layer nào mà không cần truyền qua parameters.

```
// Set context trong filter/interceptor
RequestContext context = RequestContext.builder()
    .requestId(UUID.randomUUID().toString())
    .userId("user-123")
    .workspaceId("workspace-456")
    .build();
RequestContextHolder.setContext(context);

// Access context trong service
RequestContext context = RequestContextHolder.getContext();
String userId = context.getUserId();
```

7. Ưu & Nhược điểm

- **Ưu điểm**: Code cực kỳ sạch, dễ bảo trì, unit test đạt coverage cao dễ dàng.
- **Nhược điểm**: Số lượng class/file tăng lên (Boilerplate code).
- **Khuyến nghị**: Với các service cực kỳ đơn giản (CRUD đơn thuần), có thể nới lỏng kiến trúc sang Layered để tăng tốc độ phát triển.

8. Tài liệu Tham khảo

- **MapStruct Guide**: Xem chi tiết tại `docs/MAPSTRUCT_GUIDE.md` để biết cách sử dụng MapStruct trong project.
- **Architecture Overview**: Xem `docs/standard/01. Architecture Overview.md` để hiểu tổng quan về kiến trúc hệ thống.
